

Intelligent Network Configuration Assistant

Using MCP and Rule-Based Automation | CS331

Team ID: T015

Project ID: 5 **Members:** Kshitij Srivastava 24110174, Kushal Kumar 24110178, Tejas Prasad 24110369, Vishawraj Meena 24110388, Pandit Shreyash Malhari 24110334

1. Introduction

This project implements a local network configuration assistant for Linux. Administrators use a terminal interface to request firewall changes, bandwidth limits, and other diagnostic checks. An LLM selects a constrained operation, while a deterministic Python backend retains control over validation, policy enforcement, execution, and audit logging.

2. Problem statement

Manual network administration requires command syntax knowledge and consistent checks to avoid unintended changes. Unrestricted AI-generated shell execution introduces additional risk. The project addresses this issue by separating natural language interpretation from privileged execution and evaluating the resulting backend against nonfunctional requirements.

3. Objectives

Expose ten approved MCP tools; reject invalid or policy-prohibited inputs; support dry-run and live operation; verify kernel state after changes; and retain structured audit evidence. Evaluate security controls, repeatability, concurrency, response time, and bounded failure in an isolated lab.

4. Technology background

MCP provides a client-server protocol for tool discovery and invocation [1]. Python and YAML implement the control logic. Linux iptables configures packet filtering, while tc with Hierarchical Token Bucket (HTB) controls egress bandwidth [2]. Docker provides the container network environment [3]. Ollama serves Qwen3:4b locally; ping, iperf3, pytest, and Git support validation and development.

5. System requirements

The implementation requires Fedora Linux, Python with the MCP SDK and PyYAML, Docker Engine and Compose, and the Linux network utilities. The admin container receives NET_ADMIN and NET_RAW capabilities. The workstation requires 8GB RAM and enough memory to download the 2.5 GB Qwen3:4b model. A command-line interface is used.

6. Architecture

Location	Component and responsibility
Fedora host	Terminal client exchanges requests with local Ollama/Qwen3 and calls the MCP server through Docker exec standard input/output.
Admin container	MCP tools dispatch to Python validators, YAML policies, fixed command builders, and the centralized command runner.
Linux kernel / lab	iptables and tc change admin-container network state; inspection commands, ping, and iperf3 provide validation evidence.
Persistent logs	JSONL records correlate requests, policy decisions, command results, and verification outcomes.

The internal Docker bridge uses 172.30.50.0/24: admin is 172.30.50.10, client A is 172.30.50.20, and client B is 172.30.50.30. Both clients run iperf3 servers. The admin is an endpoint, not a forwarding router so its rules do not automatically govern traffic directly between the clients.

7. Implementation

The LLM proposes a tool name and JSON arguments but it never receives a general command execution tool. The backend (python) validates IP addresses, CIDRs, ports, protocols, rates, interfaces, and traffic directions, then applies YAML restrictions. All network subprocesses pass through one runner using argument lists and shell=False, with an executable allowlist and bounded timeouts.

Live firewall operations inspect existing rules and use managed rule identifiers. Repeated requests avoid duplicate changes. Bandwidth operations use HTB classes and destination filters for traffic leaving admin toward a selected host. Dry-run returns a command plan without applying changes or claiming kernel verification. Audit records capture timestamps, request IDs, arguments, exit status, output, duration, and validation results.

8. Use cases

The ten tools are block_ip, allow_ip, block_port, allow_port, limit_bandwidth, remove_bandwidth_limit, ping_host, validate_bandwidth, show_firewall_rules, and show_bandwidth_rules. Allow operations remove matching assistant-managed DROP rules, they can't override unrelated firewall restrictions.

Example: "Block TCP port 23" becomes block_port with port=23, protocol=tcp, direction=INPUT. After validation and policy approval, Python constructs an iptables DROP rule. Live mode applies and checks the rule; dry-run reports only the plan. Both produce a structured response and an application audit record.

9. Testing and validation

Parameter	Recorded outcome
Unit correctness	121 pytest tests passed.

Parameter	Recorded outcome
Security controls	Protected-port, command-like IP, outside-lab target, and Boolean-port schema rejection checks passed.
Reliability / concurrency	Eight concurrent calls across two servers produced one change and seven no-ops; batch duration 48.93 ms. Repeat apply/remove and final cleanup passed.
Audit coverage	79 application request IDs each matched exactly one final audit record.
Bounded failure	Unreachable probe returned in 3,104.46 ms against the test budget of 12,000 ms.
Bandwidth behavior	5 Mbps requested; iperf3 measured 4.96 Mbps sender and 4.74 Mbps receiver throughput.

10. Limitations and issues resolved

Three significant issues were resolved: tc JSON verification initially rejected an applied limit because field layout and rate representation differed from assumptions. Audit correlation initially failed for all 79 requests because host and container logs were separate; persistent log sharing restored coverage. Qwen initially returned prose rather than a tool call so constrained structured output enabled the demonstrated request.

11. References

[1] Model Context Protocol. Architecture overview.

<https://modelcontextprotocol.io/docs/learn/architecture>

[2] Linux manual pages. tc-htb(8): Hierarchy Token Bucket. <https://man7.org/linux/man-pages/man8/tc-htb.8.html>

[3] Docker documentation. Networking overview. <https://docs.docker.com/engine/network/>